

Exercise 4: Routing to a Course Detail Page

Context: When Liya clicks on a course title, she should navigate to `/courses/1` and see proof that the route parameter arrived (this exercise forms your navigation spine). A rich detail screen (including title, seat counts, and HATEOAS links from `CourseDetailDto`) would use `CourseService.getById(id())` or pre-loaded data structures; that specific data layering is intentionally out of scope here so you are not debugging HTTP networking and routing engines in the same hour.

[!NOTE]

Step 1: Generate the Detail Component

Execute the Angular CLI schematic tool to scaffold a new feature routing node:

```
ng generate component features/course-detail
```

[!NOTE]

Step 2: Add the Parameterized Route

Open `src/app/app.routes.ts`. Append the route mapping configuration targeting your route constraint token:

```
{
  path: 'courses/:id',
  loadComponent: () => import('./features/course-detail/course-
detail.component')
    .then(m => m.CourseDetailComponent)
}
```

[!NOTE]

Step 3: Use Input Binding for the Route Parameter

Open `src/app/features/course-detail/course-detail.component.ts`. Map the incoming URL token parameter using modern Signal input values and a side-effect monitoring execution loop:

```
import { Component, input, effect } from "@angular/core";
import { RouterLink } from "@angular/router";

@Component({
  selector: "app-course-detail",
  standalone: true,
  imports: [RouterLink], // Required to parse routerLink directives
  in the template HTML
  templateUrl: "./course-detail.component.html",
```

```

})
export class CourseDetailComponent {
  // Automatically receives the :id parameter from the URL
  /courses/:id
  // Enabled by withComponentInputBinding() inside app.config.ts.
  Name must match exactly.
  id = input.required<string>();

  // The constructor sets up effect() to watch and execute code
  blocks every time id() emits changes
  constructor() {
    effect(() => {
      console.log(`Loading course detail for ID: ${this.id()}`);
    });
  }
}

```

Open `src/app/features/course-detail/course-detail.component.html`. Declare your navigation link anchor:

```

<h1>Course Detail</h1>
<p>Course ID: {{ id() }}</p>
<a routerLink="/dashboard">Back to Dashboard</a>

```

Navigation Philosophy: The `routerLink` attribute performs soft client-side navigation. It intercepts click behavior to update the viewport without performing a heavy browser-level page reload, keeping your local application state memory intact.

[!NOTE]

Step 4: Link from the Course Card UI

Open `src/app/ui/course-card/course-card.component.ts`. Register your router component requirements:

```

import { Component, input, output } from "@angular/core";
import { RouterLink } from "@angular/router";
import { Course } from "../../models/course.model";

@Component({
  selector: "tms-course-card",
  standalone: true,
  imports: [RouterLink], // Injects router capabilities straight
  into the card layout context
  templateUrl: "./course-card.component.html",
  styleUrls: ["./course-card.component.scss"],
})
export class CourseCardComponent {
  course = input.required<Course>();

```

```
enrollClicked = output<Course>();
}
```

Open `src/app/ui/course-card/course-card.component.html`. Wrap the header text node with dynamic parameter interpolation:

```
<h3>
  <a [routerLink]="['/courses', course().id]">{{ course().title }}
</a>
  ({{ course().code }})
</h3>
```

Template Compilation Features: The bracket syntax `[routerLink]` enforces active property binding evaluation. Passing an expression array (`['/courses', course().id]`) causes Angular to compile the path elements into valid target strings dynamically (e.g., `/courses/1`).

Troubleshooting & Common Edge Cases

Problem	Cause	Fix
Clicking the link does nothing	<code>RouterLink</code> is omitted from the component class's standalone <code>imports</code> metadata array.	Import <code>RouterLink</code> and add it directly inside the <code>@Component</code> imports block.
URL changes but the page is blank	Missing the core structural <code><router-outlet /></code> placeholder inside your root application canvas.	Open <code>app.component.html</code> and verify the template anchor tag exists.
<code>id()</code> returns <code>undefined</code>	The Signal input identifier name does not match the precise variable name declared inside the routing path template.	Rename the variable to match exactly. The route says <code>:id</code> , so your input field must be named <code>id</code> .

Checkpoint 4 Verification Checklist

- ☐ Clicking a course title redirects the viewport to `/courses/1` (or your specific entity key matching your metrics)
- ☐ The course detail text output region accurately presents the expected path index parameter value
- ☐ The soft-navigation "Back to Dashboard" anchor successfully restores the main view layout without a full page refresh